4. Bert 개념

1. BERT란

Bidirectional Encoder Representations from Transformers

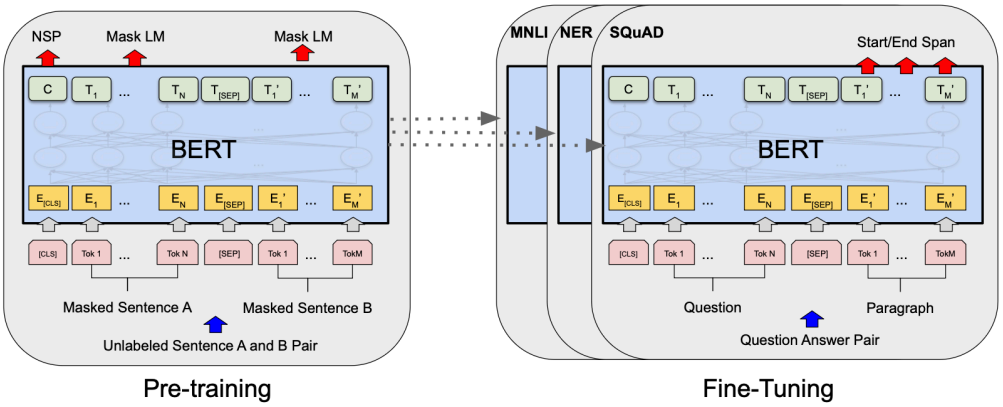
- 2018년 구글이 발표한 사전학습 언어 모델
- 기존의 많은 언어 모델은 문장을 왼쪽에서 오른쪽으로만 읽거나, 반대로 오른쪽에서 왼쪽으로만 읽는 식으로 한 방향 문맥에 더 강하게 의존했음
- 반면 BERT는 문장 안의 단어를 볼 때 **왼쪽 문맥과 오른쪽 문맥을 동시에 참고**해서 의미를 파악한다. 그래서 단어 하나를 해석할 때도 앞 단어와 뒤 단어를 함께 보며 더 풍부한 의미 표현을 만들 수 있음
- 양방향 구조 덕분에 단어의 의미뿐 아니라 **문장 전체 문맥 속에서의 역할**까지 더 잘 반영할 수 있고, 그 결과 문장 분류, 감성 분석, 질의응답, 개체명 인식 같은 다양한 자연어 처리 작업에 적용 가능한 강력한 사전학습 모델이 되었음

⇒ 대규모 텍스트로 먼저 일반적인 언어 이해 능력을 학습한 뒤 다양한 downstream task에 활용하는 기반 모델

---

2. BERT가 중요한 이유

- BERT 이전에도 Transformer 기반 언어 모델은 있었지만, BERT는 **Transformer의 인코더를 중심으로 양방향 문맥 이해를 본격적으로 밀어붙인 모델**
- GPT 계열이 디코더 중심, 즉 다음 토큰 예측 중심의 autoregressive 방식에 가깝다면, BERT는 인코더 중심으로 문장 전체를 양방향으로 보며 단어 표현을 학습함
- BERT의 강점: 문장을 읽고 그 안의 관계를 이해하는 능력에서 나오며, 그 결과 downstream task에서 별도의 큰 구조 변경 없이도 좋은 성능을 낼 수 있게 된다.



---

3. BERT의 핵심 구조: Encoder-only, 양방향 문맥 이해

1) Encoder-only Transformer

- 디코더 없이 인코더 블록만 여러 층 쌓아서 입력 문장을 처리
- 입력된 문장은 토큰 단위로 임베딩되고, 이 토큰들은 Transformer 인코더를 통과하면서 문맥이 반영된 표현으로 바뀜

2) 양방향 문맥 이해

- 각 토큰이 문장 안의 다른 모든 토큰을 양방향으로 참고
- 그래서 특정 단어의 표현은 **문장 전체를 고려한 contextualized representation**이 됨
- **Self-Attention을 통해 입력 전체를 동시에 참조**

---

4. BERT의 입력 표현: Token, Position, Segment

- BERT의 입력 임베딩의 구성: **word embeddings**, **position embeddings**, **token_type embeddings**

1) token embedding

- 문장 안의 각 토큰을 벡터로 바꾼 것으로, 기본적인 단어 의미를 담음.

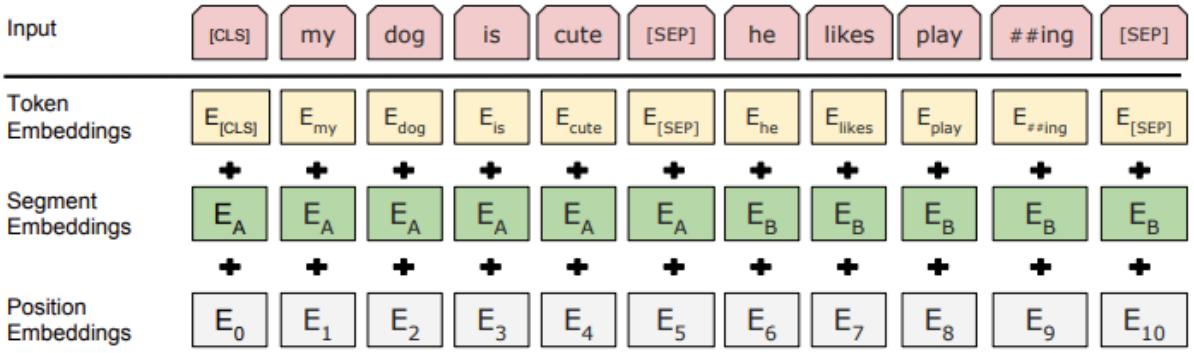
2) position embedding

- Transformer는 순환 구조가 없기 때문에 토큰 순서 정보가 자동으로 들어가지 않음
- 그래서 각 토큰이 문장 몇 번째 위치에 있는지 알려 주는 위치 임베딩이 필요

3) segment embedding(token type embedding)

- BERT가 문장 쌍을 입력으로 받는 상황에서 첫 번째 문장에 속한 토큰인지, 두 번째 문장에 속한 토큰인지 구분해 줌

⇒ 이 단어가 무엇인지 + 문장 몇 번째인지 + 어느 문장에 속하는지 모두 다룸



---

5. BERT의 특수 토큰: [CLS], [SEP], [MASK]

1) [CLS]

- 입력 문장 맨 앞에 붙는 특수 토큰
- BERT는 인코더를 통과한 뒤 각 토큰에 대해 문맥이 반영된 벡터를 얻게 되는데, 이 중 **[CLS] 토큰의 최종 출력 벡터**를 문장 전체 대표 표현처럼 사용함.
- 그래서 문장 분류, 감성 분석 같은 작업에서는 보통 [CLS]에 해당하는 출력 벡터를 분류기에 넣음.

2) [SEP]

- 문장 구분 토큰
- 문장 하나를 끝낼 때도 쓰고, 두 문장을 하나의 입력으로 넣을 때는 **문장1 [SEP] 문장2 [SEP]** 형태로 사용해 문장 경계를 표시
- 이 토큰 덕분에 BERT는 문장 쌍 입력에서 각 문장의 범위를 인식하고, 문장 간 관계를 학습하는 데 필요한 구조를 갖게 됨

3) [MASK]

- BERT 사전학습의 핵심인 **Masked Language Modeling**에 쓰이는 토큰
- 입력 문장 일부 단어를 [MASK]로 가리고, 모델이 그 위치에 원래 어떤 단어가 들어가야 하는지 예측하도록 학습

- 이 방식 덕분에 BERT는 왼쪽과 오른쪽 문맥을 동시에 참고하면서 단어 의미를 학습할 수 있음

---

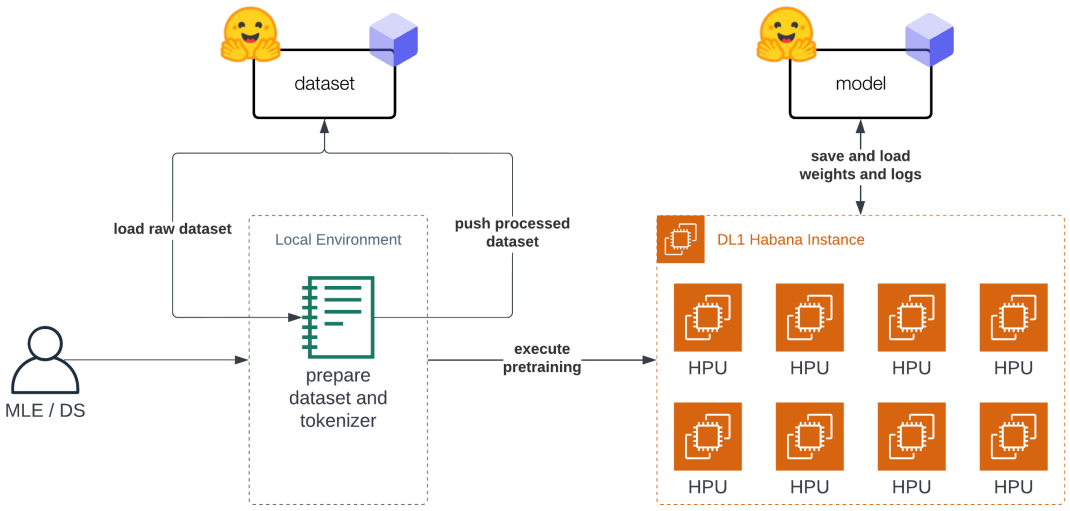
6. BERT의 사전학습 목표 1: MLM

Masked Language Modeling

- 문장 속 일부 토큰을 가리고, 그 빈칸에 원래 어떤 단어가 들어가야 하는지를 맞히게 하는 방식
- MLM이 중요한 이유
  - BERT의 양방향성을 가능하게 해 줌
  - 만약 단순히 다음 단어 예측만 한다면 모델은 현재 위치의 왼쪽 문맥에만 의존
  - 하지만 MLM에서는 문장 중간 단어를 가린 뒤 맞히게 하므로, 모델은 그 단어 앞뒤 문맥을 모두 참고해야 함

    ex) “I am [MASK] PyTorch” → [MASK]에 들어갈 단어를 맞히기 위해 앞의 `I am` 과 뒤의 `PyTorch` 를 함께 봐야 함
- MLM에서 모든 단어를 마스킹하지 않는 이유
  - 학습 시 문장 전체 토큰의 약 15%를 대상으로 삼고, 그중 80%는 실제 [MASK]로 바꾸고, 10%는 임의의 다른 단어로 바꾸고, 나머지 10%는 원래 단어를 그대로 둠

⇒ 모델이 [MASK] 토큰에만 과도하게 적응하지 않도록 하기 위한 장치



---

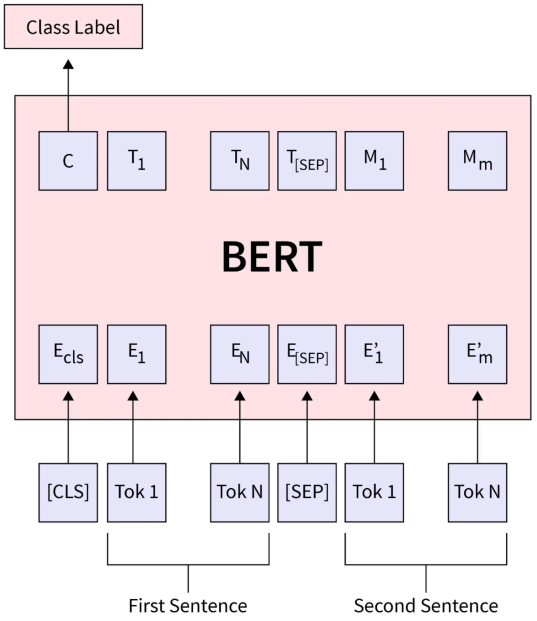
7. BERT의 사전학습 목표 2: NSP

Next Sentence Prediction

- 두 문장이 주어졌을 때, 두 번째 문장이 첫 번째 문장 바로 다음에 실제로 이어지는 문장인지 아닌지를 맞히는 작업
- BERT가 **문장과 문장 사이의 관계**도 학습하도록 만듬

`[CLS] 문장1 [SEP] 문장2 [SEP]`

- 모델은 [CLS] 출력 벡터를 이용해 두 문장이 연속 관계인지 아닌지를 판별
- 이 과정을 통해 BERT는 문장 단위의 의미 연결과 담화 흐름까지 어느 정도 반영할 수 있게 됨



---

8. BERT가 downstream task에 적용되는 방식

- 사전학습된 표현을 바탕으로 **여러 작업에 쉽게 전이**할 수 있음
- 감정 분석, 텍스트 분류, 개체명 인식, 질의응답 등 다양한 자연어 처리 작업에 BERT를 적용할 수 있음
  a. 대규모 텍스트에서 BERT를 사전학습해 일반적인 언어 이해 능력을 익힘
  b. 실제 하고 싶은 작업, 예를 들어 감성 분석이라면 문장 전체를 입력으로 넣고 [CLS] 출력 벡터를 분류기에 연결해 fine-tuning
- 질문응답이나 개체명 인식처럼 토큰 단위 출력이 필요한 작업에서는 [CLS] 하나가 아니라 **각 토큰의 출력 벡터 전체**를 활용할 수 있음

⇒ 사전학습은 공통으로 해 두고, 실제 작업에 따라 출력층만 달리 얹어 여러 문제에 재사용할 수 있음